

# 이 과정의 도착점

3일 뒤에 할 수 있는 일, 한 줄

Terminal 명령 여러 줄을 Script 파일 하나로 묶어 실행하면,  
폴더 생성부터 파일 정리·결과 기록까지 한 번에 끝나는 것을 화면에서 직접 짚어 보인다

## Day 1 · File System

directory · path · GUI vs CLI



## Day 2 · Process · 자원

Task Manager · Memory 관찰



## Day 3 · Script 자동화

명령 여러 줄 = 프로그램의 원형

오늘은 첫 칸 — 파일이 사는 세계와, os에게 말을 거는 두 가지 방법

# 관리인을 만난다 — File System과 두 개의 창구

눈에 보이는 것부터: 파일의 주소 체계, 그리고 마우스와 키보드라는 두 개의 문

## OS

부품과 프로그램 사이의 관리인

## GUI

File Explorer — 마우스로 말 걸기

## CLI

Terminal — 글자로 말 걸기

## Path

파일의 주소 — 절대 · 상대

**오늘의 제출물: GUI-CLI 비교표 1장 (워크시트 실습 9)**

같은 폴더 구조를 두 방법으로 만들어 보고, 두 창구가 같은 OS를 부른다는 것을 몸으로 확인합니다

# 오늘의 시간표

50분 수업 + 10분 휴식 · 점심 12:00~13:00 · 각 실습은 워크시트의 같은 번호 칸에 기록

**09:00** S1 · 오프닝 — 3일의 지도와 OS의 등장

**10:00** S2 · Operating System — 하드웨어와 프로그램 사이

**11:00** S3 · File System — Drive · Tree · Path

**12:00** 점심

**13:00** S4 · 확장자와 숨김 파일

**14:00** S5 · Terminal 첫 대면 — Shell · Prompt · 탐색

**15:00** S6 · CLI로 만들고 다루기

**16:00** S7 · 같은 구조를 두 번 — GUI vs CLI

**17:00** S8 · 종합 — GUI-CLI 비교표 제출

# 수업 규칙 · 워크시트 사용법

오늘의 규칙 — 예측 먼저, 화면은 검산

## 디지털 워크시트

- 예측 칸은 애니메이션·Terminal을 열기 전에 채운다
- 「채점하기」를 누르면 예측 칸이 잠기고, 틀린 칸 옆에 바른 답 표시
- 답은 브라우저에 자동 저장 — 창을 닫아도 유지
- 각 실습의 도구 ▶ 링크가 애니메이션을 새 창으로 연다
- 완료 화면에는 점수 + 내가 쓴 내용 전문이 표시

## 확인 · 제출

- 실습마다 확인 문구에 번호로 응답 (①②③)
- 실습 g(도착점) 완료 화면 = 하루 전체 요약 → 인쇄/PDF 제출
- 캡처 제출물(Terminal·비교 캡처)은 별도 채널 업로드
- 제출 채널 · 수업 규칙 상세: [강사 확정]

# 4대 부품 복습 시트

## 할 일

워크시트 실습 1 탭에서  
부품 4개의 역할·비율을 선택해 채점

## 결과물

채워진 복습 시트  
(자동 채점 8칸)

▶ 부품 복습 퍼즐

확인 “① 8칸 다 맞았다 ② 일부 틀렸다 ③ 모르겠다 — 번호로”

# 이 시간의 도착점

내 PC의 시스템 정보에서 OS 이름·버전·빌드를 찾아 옮겨 적은  
OS 정보 메모 1장을 완성한다

*S1에서 던진 질문 — “부품에게 일을 시키는 건 누구인가” — 의 답을 오늘 처음 만납니다*

# os의 정의 — 하드웨어와 프로그램 사이

## 프로그램

chrome.exe · music.exe · 여러분이 만들 프로그램

## Operating System

자원을 관리하고 프로그램에 분배하는 시스템 소프트웨어

## 하드웨어

CPU · Memory · Storage · I/O — 컴퓨터 구조에서 배운 4대 부품

*프로그램은 하드웨어를 직접 만지지 않는다 — 반드시 os를 거친다*

# os가 없다면 — Memory 충돌 실험

프로그램 두 개가 같은 Memory를 원할 때

## OS OFF

두 프로그램이 스스로 자리를 잡음  
→ 같은 칸을 덮어쓰는 →  충돌·정지

## OS ON

실행 요청 → os가 빈 구획 확인 → 서로 다른 구획 배정  
→ 둘 다 정상 동작

▶ os가 없다면 (Memory 충돌)

같은 실험을 OFF → ON 순서로 두 번 — 워크시트 실습 2 「한 줄 정리」의 재료



# OS의 4가지 관리 업무

오늘과 내일, 이 지도 위의 위치

## File System 관리

오늘

파일을 tree로 정리, path로 찾기

## Process 관리

내일

실행 중인 프로그램에 CPU 배분

## Memory 관리

내일

구획 배정과 회수

## Device(I/O) 관리

키보드 입력 → 화면 출력 중계

네 업무 모두 방금 본 3층 그림의 가운데 층에서 벌어진다

# GUI와 CLI — 같은 OS를 부르는 두 창구

## GUI — Graphical User Interface

File Explorer · 마우스 클릭  
보이는 대로 조작 — 배우기 쉬움  
반복 작업에는 손이 많이 감

## CLI — Command Line Interface

Terminal · 키보드 명령  
글자로 정확히 지시 — 처음엔 낯설  
반복·자동화·원격에 압도적

**핵심 명제 — 두 창구는 결국 같은 OS 기능을 부른다. 오늘 하루가 이 문장의 증명이다**

*개발자가 CLI를 쓰는 이유(반복·자동화)는 S7과 Day 3에서 몸으로 확인*

# 내 OS 정보 메모

## 할 일

시스템 정보(msinfo32)를 열어  
OS 이름·버전·빌드·시스템 종류를  
워크시트에 그대로 옮겨 적기

## 결과물

OS 정보 메모 1장  
(기록형 · 채점 없음)

▶ OS가 없다면 (복습용)

확인 “① 4칸 다 채웠다 ② 일부만 채웠다 ③ 창을 못 열었다 — 번호로”

다음 시간

# 관리인의 창고에는 주소가 있다

## S3 · File System — Drive · Tree · Path

모든 파일에는 주소가 있습니다. 그 주소를 읽는 법과 쓰는 법 — 오후 Terminal의 전 재료가 이 시간에 만들어집니다.

# 이 시간의 도착점

**Directory Tree만 보고 절대 경로 3개 + 상대 경로 1개를 손으로 적은  
Path 워크시트 1장을 완성한다**

*채점 후 Tree 탐험기에서 같은 파일을 클릭해 내 답과 화면의 path를 대조합니다 — 예측 먼저, 화면은 검산*

# File System의 뼈대 — Drive · Root · Tree

```
C:\ (Drive · Root)
├─ Users
│   └─ student (Home Directory)
│       ├── Documents
│       │   ├── report.txt
│       │   └─ memo.txt
│       ├── Downloads
│       └─ Pictures — trip — sea.jpg
└─ Windows — notepad.exe
```

## Drive와 Root

C:\ — 창고의 대문. 모든 주소가 여기서 출발

## Directory = 폴더

CLI에서는 directory라 부름 — 같은 것

## Home Directory

C:\Users\<사용자명> — 이후 모든 실습의 기준점

# 절대 경로 — Root부터 전체 주소

Absolute Path

**C:\Users\student\Documents\report.txt**

- Root(C:\)에서 파일까지 가는 길을 separator(\)로 이어 적는다
- 어디서 읽어도 같은 파일을 가리킨다 — 전국 어디서나 통하는 도로명 주소
- 규칙: Drive부터 시작 · directory 이름을 순서대로 · 마지막이 파일명

▶ Tree **탐험기**

노드를 클릭할 때마다 상단 path가 실시간으로 조립됩니다

# 상대 경로 — 지금 위치가 기준

Relative Path · Working Directory · ..

## 지금 위치 (Working Directory)

C:\Users\student\Downloads

여기서 report.txt로 가려면?

**..\Documents\report.txt**

.. = 상위 directory로 한 칸

. = 지금 위치

같은 파일이라도 기준이 바뀌면 주소가 바뀐다 — 이것이 상대 경로의 전부

▶ 주소 조립 게임 (절대→상대)



# Separator와 표기 규칙

os마다 다른 길 표시

## Windows

C:\Users\student\...

역슬래시 \ · Drive 문자(C:)

## macOS · Linux

/Users/student/...

슬래시 / · Drive 문자 없이 /에서 시작

- 이름에 쓸 수 없는 문자: \ / : \* ? " < > |
- 공백이 든 이름은 따옴표로 감싼다 — "My Folder" (S6에서 실험)
- 대소문자 구분 여부는 os마다 다름 — Windows 기본은 구분하지 않음

# Path 작성

## 할 일

워크시트의 Tree만 보고  
절대 경로 3개 + 상대 경로 1개를  
손으로 적고 「채점하기」 (잠김)

## 결과물

Path 워크시트 1장  
(자동 채점 4칸)

▶ Tree 탐험기 (검산)

▶ 주소 조립 게임

확인 “① 4칸 다 맞았다 ② 일부 틀렸다 ③ 모르겠다 — 번호로”

다음 시간

# 이름 뒤 글자가 말해주는 것

## S4 · 확장자와 숨김 파일

확장자는 파일의 내용이 아니라 「해석 방법」을 바꿉니다 — 그리고 문서로 위장한 실행 파일을 잡아내는 법까지.

# 이 시간의 도착점

파일 5개의 확장자·형식을 예측 채점하고, 내 PC에서 확장자 표시를 켜  
「여는 프로그램」까지 채운 확장자 분류표 1장을 완성한다

핵심 명제 — 확장자는 내용이 아니라 「해석 방법」을 바꾼다

# 파일 이름의 구조 — 이름.확장자

report.txt

## 문서

.txt .pdf .hwp

## 이미지

.jpg .png .gif

## 미디어

.mp3 .mp4

## 압축.실행.코드

.zip .exe .py 예고

확장자가 결정하는 것 — 이 파일을 어떤 프로그램으로 열(해석할) 것인가

# 확장자를 바꾸면 — 내용은 고정, 해석만 변경

## 고정된 것

파일의 실제 내용(바이트)

48 65 6C 6C 6F 20 4F 53 21

— 어떤 확장자로 바뀌어도 그대로

## 바뀌는 것

OS가 연결하는 프로그램

.txt→메모장 정상 · .jpg→해석 실패

.zip→손상 판정 · .exe→실행 거부

▶ 확장자 룰렛

4가지 판정을 전부 돌려 봅니다 — 실패 판정이 곧 증거

# 위장 파일 — 이중 확장자의 함정

확장자 표시를 켜야 하는 이유

이력서.pdf.exe

- 확장자 숨김(기본값) 상태에서는 「이력서.pdf」로만 보인다 — 아이콘까지 위장 가능
- 진짜 확장자는 맨 끝 — .exe면 실행 파일. 더블클릭 = 실행
- 대응: File Explorer에서 「파일 확장명」 표시 켜기 — 오늘 실습에서 실제로 켜다

▶ 위장 파일 찾기

1 단계는 감으로(대부분 실패하도록 설계) → 2 단계는 표시 켜고

# 확장자 분류표

## 할 일

파일 5개의 확장자·형식을 예측 채점(잠김)  
→ 내 PC에서 확장자 표시를 켜고  
「여는 프로그램」 칸을 실물로 확인해 기록

## 결과물

확장자 분류표 1장  
(채점 10칸 + 기록 5칸)

▶ 확장자 룰렛

▶ 위장 파일 찾기

확인 “① 표시 켜고 다 채웠다 ② 예측만 했다 ③ 설정을 못 찾겠다 — 번호로”



다음 시간

# 이제 글자로 말을 겁니다

## S5 · Terminal 첫 대면 — Shell · Prompt · 탐색

오전에 익힌 주소(path)를 그대로 들고 갑니다 — cd는 그 주소를 글자로 말하는 것뿐입니다.

# 이 시간의 도착점

탐색 명령 4개를 기억으로 적어 채점하고, 가상 Terminal 미션 3곳을 방문한 뒤  
실제 Terminal의 Prompt를 옮겨 적은 탐색 기록 1장을 완성한다

*명령어 표기는 CMD 기준 (ls·pwd도 정답 처리) — [Shell 확정 시 고정]*

# Terminal과 Shell — 창과 통역

## ① Terminal (창)

글자를 받고 보여주는 화면 — 입출력 창구

## ② Shell (통역)

입력된 글자를 「명령」으로 해석 — CMD · PowerShell · bash

## ③ OS (실행)

해석된 요청을 File System에서 실제로 수행

*dir 한 줄 → 창을 지나 → 통역이 해석 → OS가 수행 → 결과가 역순으로 화면에*

# Prompt 읽는 법 — 화면이 먼저 말을 건다

```
C:\Users\student> █
```

- Prompt 자체가 현재 Working Directory를 보여준다 — 나는 지금 어디에 서 있는가
- > 뒤가 내가 입력하는 자리 — Enter로 전송
- Tab = 자동완성 · ↑ = 이전 명령 다시 — 오타를 줄이는 두 습관
- 명령의 일반 구조: 명령어 [옵션] [인자] — 앞으로 만날 모든 명령을 읽는 틀

# 탐색 3종 — 묻고, 보고, 움직인다

## **cd (인자 없이)**

지금 어디인가 — 현재 위치 확인 (pwd)

## **dir**

여기 뭐가 있나 — 목록 보기 (ls)

## **cd <path>**

저기로 가자 — 절대·상대 경로 그대로 · cd .. 상위로

▶ [가상 Terminal \(미션 3곳\)](#)

▶ [에러 진단기 \(5문항\)](#)

에러 점검 루틴: *오타·대소문자* → *separator(\)* → *공백·따옴표*

# Terminal 탐색 기록

## 할 일

- ① 목적 4개 → 명령어를 기억으로 적고 채점(잠김)
- ② 가상 Terminal에 직접 입력해 미션 3곳 방문
- ③ 실제 Terminal을 열어 내 Prompt를 옮겨 적기

## 결과물

탐색 기록 1장 + 탐색 화면 캡처 1장  
(캡처는 별도 채널 제출)

▶ [가상 Terminal](#)

▶ [에러 진단기](#)

**확인** “① 미션 3곳 다 갔다 ② 일부만 ③ 명령이 안 먹는다 — 번호로”

다음 시간

# 보는 것에서, 바꾸는 것으로

## S6 · CLI로 만들고 다루기 — 생성·복사·이동·삭제

조작 명령 5개 — 그리고 휴지통 없이 사라지는 del의 경고까지.

# 이 시간의 도착점

작업 5개 → 명령 한 줄을 손으로 적어 채점하고,  
명령 미리보기로 결과를 대조한 CLI 조작 기록을 완성한다

현재 위치 *C:\work* 기준 · 미리보기(점선) → 실행(실체화) 2단계 습관



## 조작 5종 — 만들고 · 복사하고 · 옮기고 · 바꾸고 · 지운다

```
mkdir backup
```

directory 생성

```
copy report.txt backup\
```

복사 — 원본 유지

```
move photo.jpg backup\
```

이동 — 원본 사라짐

```
ren report.txt final.txt
```

이름 변경 — 자리 이동 없는 move

```
del final.txt
```

삭제 —  $\Delta$  휴지통 없이 즉시

▶ 명령 미리보기 (5개 탭)

탭 ①~⑤ 를 워크시트 순서 그대로 재생 — move와 ren 이 같은 원리임을 대조

# 공백과 따옴표 — Shell이 자르는 순간

## 따옴표 없이

`mkdir My Folder`

→ 토큰: `[mkdir][My][Folder]`

→ directory 2개가 생겨 버림 X

## 따옴표 씌우고

`mkdir "My Folder"`

→ 토큰: `[mkdir][My Folder]`

→ directory 1개 정상 ✓

Shell은 공백을 인자 구분자로 읽는다 — 따옴표가 공백을 묶어 하나의 인자로

▶ [따옴표 실험실](#)

# del의 경고 — 휴지통이 없다

## GUI 삭제

파일 → 휴지통으로 이동  
복원 가능 — 되돌릴 수 있는 삭제

## CLI del

휴지통을 그대로 통과 → 즉시 소멸  
복원 경로 없음

그래서 습관 — del 전에 dir로 대상 확인, 그리고 미리보기로 결과 예측

# CLI 조작 — 명령 작성

## 할 일

작업 5개(생성·복사·이동·이름변경·공백이름)  
→ 명령 한 줄씩 손으로 적고 채점(잠김)  
→ 명령 미리보기로 ①~⑤ 순서대로 대조

## 결과물

CLI 조작 기록  
(자동 채점 5줄 + 서술 1)

▶ [명령 미리보기](#)

▶ [따옴표 실험실](#)

**확인** “① 5줄 다 맞았다 ② 일부 틀렸다 ③ 따옴표가 헛갈린다 — 번호로”

다음 시간

# 같은 결과물, 두 개의 길

## S7 · 같은 구조를 두 번 — GUI vs CLI

오전의 GUI와 오후의 CLI가 같은 과제 위에서 만납니다 — 시간을 재고, 100개라면 어떨지 상상하는 것까지.

# 이 시간의 도착점

과제 구조를 GUI로 1회 · CLI로 1회 만들고, 소요 시간과 사용 명령을 기록한 기록지(캡처 2장 포함)를 완성한다

*채점은 내가 기록한 명령 기준 — 과제 4요소 포함 여부를 자동 확인*

# 과제 구조

이 구조를 두 번 만듭니다 . [과제 구조는 강사 확정 시 교체]

```
work
├── docs
├── photos
└── backup
    └── report.txt (복사본)
```

## 1회차 — GUI

File Explorer만으로 · 소요 시간 기록

## 2회차 — CLI

Terminal 명령만으로 · 사용한 명령을 실행 순서대로 기록

완성 기준 — `dir` 출력과 File Explorer 화면이 동일할 것 (비교 캡처 2장)

# 시범 경주 — 같은 결과물, 다른 과정

## GUI 선수

클릭 17회 · 단계 8개  
마우스가 개수만큼 반복

## CLI 선수

명령 4줄  
mkdir ×3 + copy ×1

▶ 동시 재생 경주

결과물은 동일 — 과정의 차이를 눈으로 확인한 뒤 내 경주를 시작합니다



# 같은 구조를 두 번 — 기록지

## 할 일

GUI 1회차 → CLI 2회차로 과제 구조 제작  
소요 시간 · 사용 명령을 기록하고 채점  
(명령 기록에서 과제 4요소 자동 검사)

## 결과물

기록지 1장 + 결과 캡처 2장  
(캡처는 별도 채널 제출)

▶ [동시 재생 경주](#)

▶ [100배 스케일](#)

**확인** “① 두 번 다 완성 ② CLI가 막힌다 ③ GUI만 했다 — 번호로”

다음 시간

# 하루를 한 장으로

## S8 · 종합 — GUI-CLI 비교표 제출

실습 1~8의 재료가 비교표 한 장으로 수렴합니다 — 그리고 짝에게 1분 설명이 오늘의 도착점.

# 이 시간의 도착점

GUI 동작 ↔ CLI 명령 대응표를 완성하고, 짝에게 1분 설명을 마친 뒤  
도착점 완료 화면(하루 전체 요약)을 만들어 제출한다

완료 화면 = 점수 + 내가 작성한 내용 전문 — 인쇄/PDF로 제출

## 대응표 — 같은 일, 두 이름

새 폴더 만들기



`mkdir`

복사 → 붙여넣기



`copy`

잘라내기 → 붙여넣기



`move`

이름 바꾸기



`ren`

삭제 (휴지통 없이!)



`del`

▶ 대응표 뒤집기 (최종 점검)

## 오늘의 도착점 확인

같은 directory 구조를 File Explorer와 Terminal 양쪽에서 각각 만들고,  
두 방법의 결과가 동일함을 확인한 GUI-CLI 비교표를 완성해 제출한다

*아침에 약속한 이 문장을 방금 해냈습니다*

다음 시간

# 오늘 만든 파일이, 실행되는 순간

## Day 2 · Process와 자원 — 관리인의 장부를 읽는다

더블클릭 한 번에 무슨 일이 벌어지는가 — Task Manager에서 CPU·Memory 숫자가 움직이는 것을 직접 관찰하고 개입합니다.

# 확인 퀴즈 1

Q C:\Users\student\Documents\report.txt — 이 표기의 이름은?

① 상대 경로

② 절대 경로

③ 확장자

④ Prompt

## 확인 퀴즈 2

Q GUI의 「잘라내기 → 붙여넣기」와 같은 일을 하는 CLI 명령은?

① copy

② del

③ move

④ ren



## 확인 퀴즈 3

Q 파일 이름 뒤 확장자가 결정하는 것은?

① 여는 프로그램

② 파일의 내용

③ 파일의 크기

④ 저장 위치